Tea Covey
6/15/2026
Foundations of Python Programming
https://github.com/teacovey/IntroToProg-Python-Mod06

Assignment 06: Use of functions, classes, and using the separation of concerns pattern

Introduction

In module 06, we learned about organizing your code by adding a section for function definitions and a section for classes, using global and local variables, using docstrings to describe classes and functions, understanding the separations of concerns (SoC) pattern and organizing code based on concerns.

What we're doing in this assignment: *Create a Python program that demonstrates using constants, variables, and print statements to display a message about a student's registration for a Python course.*

What changed from last assignment: *This program is very similar to Assignment05, but It adds* ***the use of functions, classes, and using the separation of concerns pattern.***

What we will discuss in this document is docstrings, classes, functions, global versus local variables, and parameters.

Docstrings

A docstring is short for "document string" and is used in Python as a descriptive header at the beginning of a class or function. Docstrings explain what the code does, list parameters and return values, and include a change log.

For each class and function in our assignment, a docstring is included.

When you add a docstring in an integrated development environment (IDE) like PyCharm to a class or function, it allows you to hover over the code that calls to the class or function and it will populate the docstring in a box (Mod06-Notes.pdf, page 30). This makes it easier to understand code without searching through the file or analyzing lines of code. An example of this is shown in Figure 1.1 below.

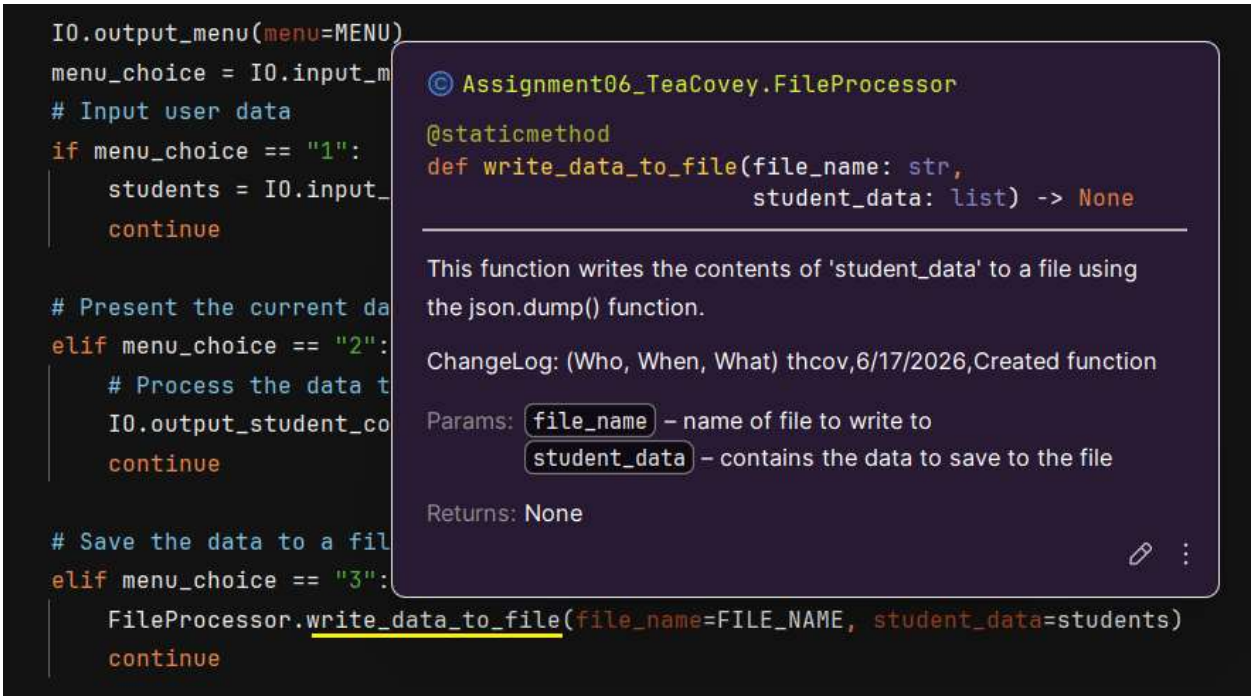

Figure 1.1: When hovering over "write_data_to_file", the docstring box appears.

The docstring for functions describes what the code does, includes a change log (who, when, what), and identifies all that are applicable: argument(s)/parameter(s), return value(s), and return type(s).

The first line following the three quotation marks is where you describe what the code does.

Next, include a change log for the function.

Finally, complete the applicable data points: parameter(s), parameter type, return value, and return type.

In PyCharm, you can generate a docstring function outline by entering three quotation marks and hitting enter. The code below is what was generated following the function write_data_from_file:

```
@staticmethod
def write_data_to_file(file_name: str, student_data: list):
    """

    :param file_name:
    :type file_name:
    :param student_data:
    :type student_data:
    :return:
    :rtype:
    """
```

Based on the data points PyCharm believes this function has, it populated a docstring outline to be filled in.

It determined that parameters were present, so it included a line for each parameter and a corresponding line for each parameter type.

It also populated lines for the return value and return type. In this case, return will be None so we can remove the return type line.

We then add the description, change log, parameter details and return information.

```
@staticmethod
def write_data_to_file(file_name: str, student_data: list):
    """This function writes the contents of 'student_data' to a file
    using the json.dump() function.

    ChangeLog: (Who, When, What)
    thcov,6/17/2026,Created function

    :param file_name: JSON file to write the data to
    :type file_name: string
    :param student_data: contains the data to save to the file
    :type student_data: list of dict. rows
    :return: None"""
```

The above docstring is an example of how the docstring could be completed for this function.

Functions

Functions are one of the ways in which we can organize our code. “In programming, a function is a reusable block of code that performs a specific task or a set of tasks” (Mod06-Notes.pdf, page 2).

For this assignment, we created the following functions:

1. output_error_messages(message: str, error: Exception = None)
2. output_menu(menu: str)
3. input_menu_choice()
4. output_student_courses(student_data: list)
5. input_student_data(student_data: list)
6. read_data_from_file(file_name: str, student_data: list)
7. write_data_to_file(file_name: str, student_data: list)

This list of functions is a great example to reference back to on how to break out your code into appropriate functions. Each of the above functions groups code that executes one task (i.e. output_error_messages executes the task of displaying error message(s) to the user).

> *"Functions allow you to group a set of programming statements and later reference them by a given name. In Python, you place you[sic] functions after the variable declarations since they must be defined in a script before they can be called. The area after the functions are defined is called the 'main body' of the script. The statements within the function run later in your program by 'calling' the function from the main body"* (Mod06-Notes.pdf, page 3).

By moving blocks of code into the functions definitions of the script, it simplifies our code, making it more readable and manageable.

```
# Beginning of main body of the script
# When the program starts, read the file data into a list of lists (table)
# Extract the data from the file
students = data_from_file(file_name=FILE_NAME, student_data=students)

# Present and Process the data
while True:
    # Present the menu of choices
    output_menu(menu=MENU)
    menu_choice = input_menu_choice()
    # Input user data
    if menu_choice == "1":  # This will not work if it is an integer!
        students = input_student_data(student_data=students)
        continue

    # Present the current data
    elif menu_choice == "2":
        # Process the data to create and display a custom message
        output_student_courses(students)
        continue

    # Save the data to a file
    elif menu_choice == "3":
        write_data_to_file(file_name=FILE_NAME, student_data=students)
        continue

    # Stop the loop
    elif menu_choice == "4":
        break  # out of the loop

print("Program Ended")
```

Next, we will learn how to organize this code further into classes.

Classes

Next, we implemented the use of classes to further organize our code. Using classes is a way to organize complex code by grouping functions into different classes based on the type of tasks

executed. “Classes provide a natural way to organize code by grouping functions and data needed by those functions, making code more structured and readable, especially in larger projects.” (Mod06-Notes.pdf, page 27)

The two classes used in this script are called *FileProcessor* and *IO*. We moved all of our functions we created into these two classes.

FileProcessor holds only two functions in its class:

- read_data_from_file(file_name: str, student_data: list)
- write_data_to_file(file_name: str, student_data: list)

These functions are different from all other functions in this script because they relate to processing data to and from the file.

IO is short for “input/output.” The functions that fall under the *IO* class include:

- output_error_messages(message: str, error: Exception = None)
- output_menu(menu: str)
- input_menu_choice()
- output_student_courses(student_data: list)
- input_student_data(student_data: list)

The *IO* class includes all functions that are related to presentation (inputting or outputting data from or to the user).

Global v. local variables

An important concept to understand with the implementation of classes and functions is how variables operate differently when used in a function or class. The idea of global versus local variables helps us understand these differences.

What are global variables? “Global Variables: Variables declared outside of any function, typically in the main body of the script, are considered global to the entire script. These variables can be accessed and modified from anywhere within the script. They are ‘in scope’ throughout the entire script” (Mod06-Notes.pdf, page 7).

We initially had the below variable defined at the top of our script (outside our classes/functions section):

```
file = None
```

As a global variable, this means that this definition of *file* continues through the entire script.

When we used this code inside our *read_data_from_file* function, we use the “global” keyword to reference the *file* variable within the function, to clarify that the use of the “file” variable is a reference to the existing global variable and not a different “local variable.”

```
global file
```

“Local Variables: Variables declared within a function are considered local to that function. These variables can only be accessed and used within the function where they are defined. They are ‘inside of the function's scope’” (Mod06-Notes.pdf, page 7).

However, the global variable only is needed within the *read_data_from_file* function and the *write_data_to_file* function. Instead of calling to a global variable, we can remove the global variable from the script and create local variables in these functions, defining *file*. This means that the variable would only exist within the scope of each of these functions, but because *file* is not used elsewhere throughout the script, it cleans up our code by moving the global variable into the functions as local variables!

```
@staticmethod
def read_data_from_file(file_name: str, student_data: list):
    file = None

    try:
        file = open(file_name, "r")
        student_data = json.load(file)
    except Exception as e:
        message = "Error: There was a problem loading the file data into the
program.\n"
        message += "Please check to make sure the file exists and is a json
file."
        IO.output_error_messages(message=message,error=e)
    finally:
        if file is not None and file.closed == False:
            file.close()
    return student_data
```

Another important note: “In Python, when you reference a variable inside a function without explicitly declaring it as a local variable, the interpreter will automatically consider it a reference to the global variable with the same name.” (Mod06-Notes.pdf, page 7).

Parameters

In this assignment, we used parameters to pass in variables to our functions. Using parameters is a preferable method to using global variables. “Using parameters is a best practice because it promotes encapsulation, reusability, and predictable behavior in your functions. It also reduces the reliance on global data, which can lead to cleaner and more maintainable code” (Mod06-Notes.pdf, page 15).

Before the implementation of “parameters,” our code for the *write_data_to_file* function would look like the below:

```
@staticmethod
def write_data_to_file():
    file = None
    global students
    global FILE_NAME

    try:
        file = open(FILE_NAME, "w")
        json.dump(students, file, indent=2)
    except TypeError as e:
        print("Please check that the data is a valid JSON format \n")
        print(" -- Technical Error Message -- ")
        print(e, e.__doc__, type(e), sep='\n')
    except Exception as e:
        print(" -- Technical Error Message -- ")
        print(e, e.__doc__, type(e), sep='\n')
    finally:
        if file is not None and file.closed == False:
            file.close()
```

And below would be the call to *write_data_to_file*:

```
elif menu_choice == "3":
       FileProcessor.write_data_to_file()
       continue
```

“Instead of accessing data using global variables in your functions, you can pass the data into your function by creating parameters” (Mod06-Notes.pdf, page 14).

With the use of parameters, we are able to pass in variables that act as local variables just for the operation of the function. Using parameters, the code is updated below.

```
@staticmethod
def write_data_to_file(file_name: str, student_data: list):
    file = None

    try:
        file = open(file_name, "w")
        json.dump(student_data, file, indent=2)
        IO.output_student_courses(student_data=student_data)
    except Exception as e:
        message = "Error: There was a problem with writing to the file.\n"
```

```
        IO.output_error_messages(message=message,error=e)
    finally:
        if file is not None and file.closed == False:
            file.close()
```

We've updated the function to pass in *file_name* and *student_data* in place of *FILE_NAME* and *students*, respectively. Therefore, we have updated the related references to reflect the new variables that act as local variables within these functions.

```
elif menu_choice == "3":
    FileProcessor.write_data_to_file(file_name=FILE_NAME, student_data=students)
    continue
```

In the updated call to the *write_data_to_file* function (shown above), we pass in the same variables but also assign them to their related global variable (i.e. file_name=FILENAME).

Summary

In this assignment, we drastically changed the appearance of our code by organizing functions into related classes and making calls to those functions in the body of our script. Additionally, we added docstrings to the addition of each "class" and "function" in our script. Finally, we had to distinguish between the variables and constants we used throughout our script in a different way because of the idea of global v. local variables in coding.